

Basic (a normal, exact test) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper pytest), **Advanced** (how a real test suite catches the edge cases a hand-picked example misses). Read Basic first even if you already know pytest — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

`chunk_by_paragraph` is a plain Python function — no LLM involved. That means you can test it the normal way: give it a known input, and check the output is exactly what you expect.

A pytest test is just a function whose name starts with `test_`. Inside it, you use plain `assert` statements. If the assert is true, the test passes silently. If it's false, pytest shows you exactly what didn't match.

Think about what you already know for sure: if you feed the function a piece of text with 3 paragraphs separated by blank lines, you already know the answer should be 3 chunks — before you even run the code.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

This exercise is really about the difference between testing *deterministic* code (same input, same output, every time) and the LLM-touching tests later in this document. `chunk_by_paragraph` has no model call inside it, so an exact `assert` is completely appropriate here — you should not reach for anything fuzzier.

pytest auto-discovers any function starting with `test_` in any file starting with `test_`. It needs no special setup beyond that naming convention. A bare `assert expression` is the whole mechanism — pytest rewrites it under the hood to show you a helpful diff when it fails, without you writing any extra code.

The exact pieces:

- Save your test in a file named `test_chunking.py` (pytest needs the `test_` prefix on the filename too).
- Import the function you're testing: `from chunking import chunk_by_paragraph`.
- Write one sample string with a couple of blank lines in it, to mark paragraph breaks.
- Run it with: `pytest test_chunking.py -v`

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

One hand-picked example (3 neat paragraphs, clean blank lines) only proves the function works on the one input you thought of. A real chunker gets fed messy text: a single paragraph with no blank lines at all, a string with 3+ blank lines in a row between paragraphs, leading/trailing blank lines, or an empty string.

The dangerous failure mode isn't a crash — it's **silent data loss**. An off-by-one in the split logic can quietly drop the last sentence of the last paragraph, and a test that only checks `len(chunks) == 3` would never notice, because the count would still look right in your one example while being wrong on a different one.

The real design question: instead of picking one more example and asserting its exact output, can you write a check that holds for *any* input — something like "every word in the original text shows up somewhere in the chunks"? That's a property, not a specific example, and it's what a hand-written edge case can't fully cover on its own.

The extra pieces needed:

- `@pytest.fixture` — a function decorated with this that returns your sample text, so multiple test functions can reuse it without copy-pasting the same string.
- `@pytest.mark.parametrize` — runs the same test body against a table of different inputs (empty string, single paragraph, many blank lines), each reported as its own pass/fail.
- A **no-content-lost** check: something like `assert all(word in " ".join(chunks) for word in original_text.split())` — this catches silent truncation that a count-only assert would miss.
- `hypothesis` (a real property-based testing library, `pip install hypothesis`) — instead of you inventing edge cases by hand, it generates hundreds of random strings and tries to find one that breaks your no-content-lost property. Worth knowing exists, even if you don't reach for it on a function this small.

Sketch one no-content-lost assertion yourself before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic describes the plain idea and lists the tools for one tidy, hand-picked example. Intermediate shows the exact pytest naming/import mechanics for that same one example. Advanced steps back and asks what a single example *can't* prove — that nothing gets silently dropped on messy, unplanned input — and introduces `parametrize`, fixtures, and a content-preservation property as the way to cover that gap instead of hand-picking more one-off examples forever.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
import chunk_by_paragraph from chunking

make sample_text = a string with 3 paragraphs, separated by blank lines

def test_chunk_by_paragraph():
    chunks = chunk_by_paragraph(sample_text)
    assert there are exactly 3 chunks
```

Here's almost the whole thing — just try running it and reading it line by line:

```
from chunking import chunk_by_paragraph

sample_text = "Paragraph one.\n\nParagraph two.\n\nParagraph three."

def test_chunk_by_paragraph():
    chunks = chunk_by_paragraph(sample_text)
    assert len(chunks) == 3
```

Expected output: run with `pytest test_chunking.py -v` — you should see `test_chunk_by_paragraph PASSED`. If instead you see nothing at all was collected, check the filename starts with `test_` and the function name starts with `test_`.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
from chunking import chunk_by_paragraph

sample_text = "Paragraph one.\n\nParagraph two.\n\nParagraph three."

def test_chunk_by_paragraph():
    chunks = chunk_by_paragraph(sample_text)
    assert len(chunks) == 3
    assert chunks[0] == "Paragraph one."
    assert chunks[-1] == "Paragraph three."
```

```
from chunking import chunk_by_paragraph

sample_text = "Paragraph one.\n\nParagraph two.\n\nParagraph three."

def test_chunk_by_paragraph():
    chunks = chunk_by_paragraph(sample_text)
    assert len(chunks) == 3
    assert chunks[0] == "Paragraph one."
    assert chunks[-1] == "Paragraph three."
```

Notice the second and third asserts — checking the count alone would still pass even if the chunker scrambled the paragraph order or dropped their text. Now write a second test function covering an edge case — a string with only one paragraph and no blank lines at all — before moving to Advanced.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

make a fixture `sample_text()` that returns the 3-paragraph string

make a table of (text, expected_count) cases:

```
the 3-paragraph string -> 3
a single paragraph, no blank lines -> 1
an empty string -> 0
"A.\n\n\nB." (extra blank lines) -> 2
```

parametrize a test over that table:

```
chunks = chunk_by_paragraph(text)
assert len(chunks) == expected_count
```

separately, define `test_no_content_lost(sample_text)`:

```
chunks = chunk_by_paragraph(sample_text)
rejoined = " ".join(chunks)
for word in sample_text.split():
    assert word in rejoined
```

Here's almost the whole thing — fill in the missing case yourself:

```
import pytest
from chunking import chunk_by_paragraph

@pytest.fixture
def sample_text() -> str:
    return "Paragraph one.\n\nParagraph two.\n\nParagraph three."

@pytest.mark.parametrize(
    "text,expected_count",
    [
        ("Paragraph one.\n\nParagraph two.\n\nParagraph three.", 3),
        ("Just one paragraph, no blank lines.", 1),
        ("", 0),
        # your turn: add a case with extra blank lines between paragraphs,
        # e.g. "A.\n\n\n\nB." — decide what count YOUR chunker should give it
    ],
)
```

```
def test_chunk_by_paragraph_count(text: str, expected_count: int) -> None:
    chunks = chunk_by_paragraph(text)
    assert len(chunks) == expected_count

def test_no_content_lost(sample_text: str) -> None:
    chunks = chunk_by_paragraph(sample_text)
    rejoined = " ".join(chunks)
    for word in sample_text.split():
        assert word in rejoined
```

Fill in the missing `parametrize` case and run all of this, then compare against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both check one hand-picked example, at increasing levels of assert detail (count only, then count plus first/last chunk content). Advanced replaces "pick one more example" with a table of edge cases run through the same test body via `parametrize`, plus a fixture so the sample text isn't retyped in every function, plus a separate property test (`test_no_content_lost`) that would catch a silent truncation bug no exact-match example happens to expose. That property test is the direct answer to Hint 1's Advanced question — a check that holds for many inputs, not just the one you picked.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)